

DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT

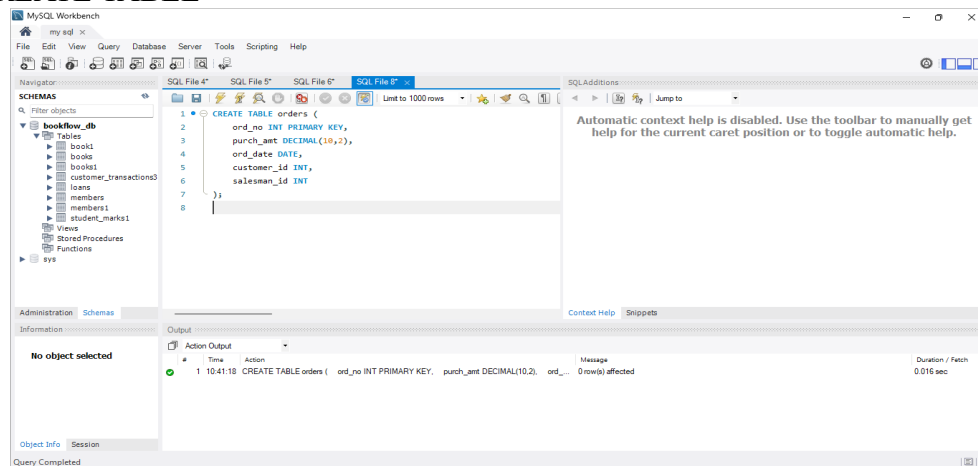
Sec:- 3

Name:- Srithika Kambhammettu

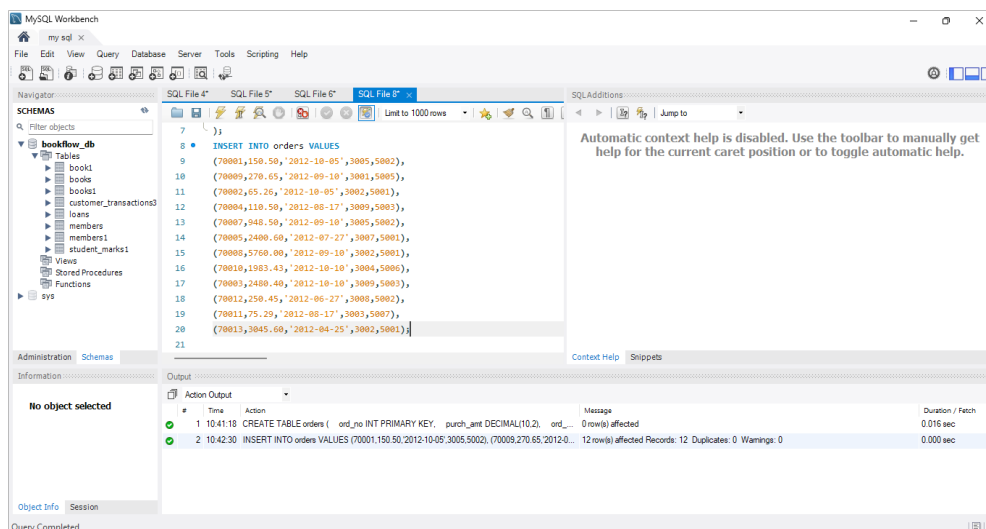
Roll no:- 2520030361

2- WHERE, ORDER BY, GROUP BY, HAVING & AGGREGATE FUNCTIONS

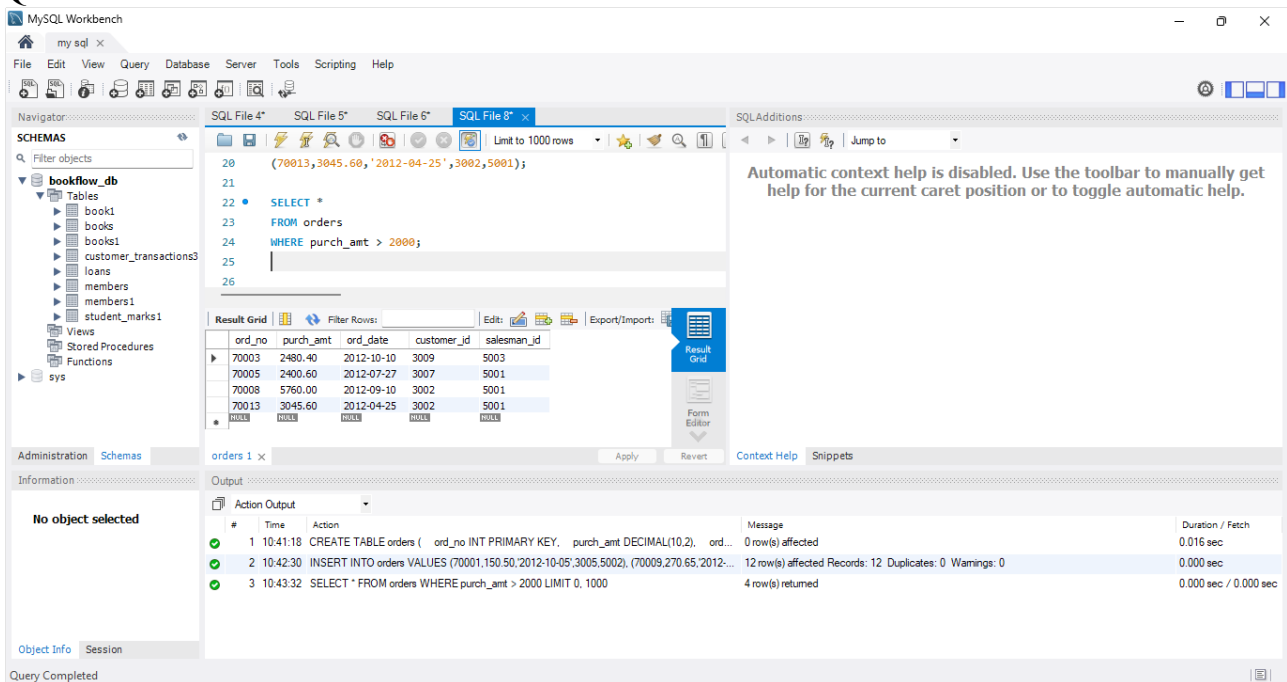
STEP 1: CREATE TABLE



STEP 2: INSERT DATA



QUESTION 1 - WHERE CLAUSE



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'bookflow_db' schema with tables like 'book1', 'books', 'books1', 'customer_transactions3', 'loans', 'members', 'members1', 'student_marks1', 'Views', 'Stored Procedures', and 'Functions'. The main editor window shows a SQL query in 'SQL File 8*':

```
20 (70013,3045.60,'2012-04-25',3002,5001);
21
22 • SELECT *
23 FROM orders
24 WHERE purch_amt > 2000;
25
26
```

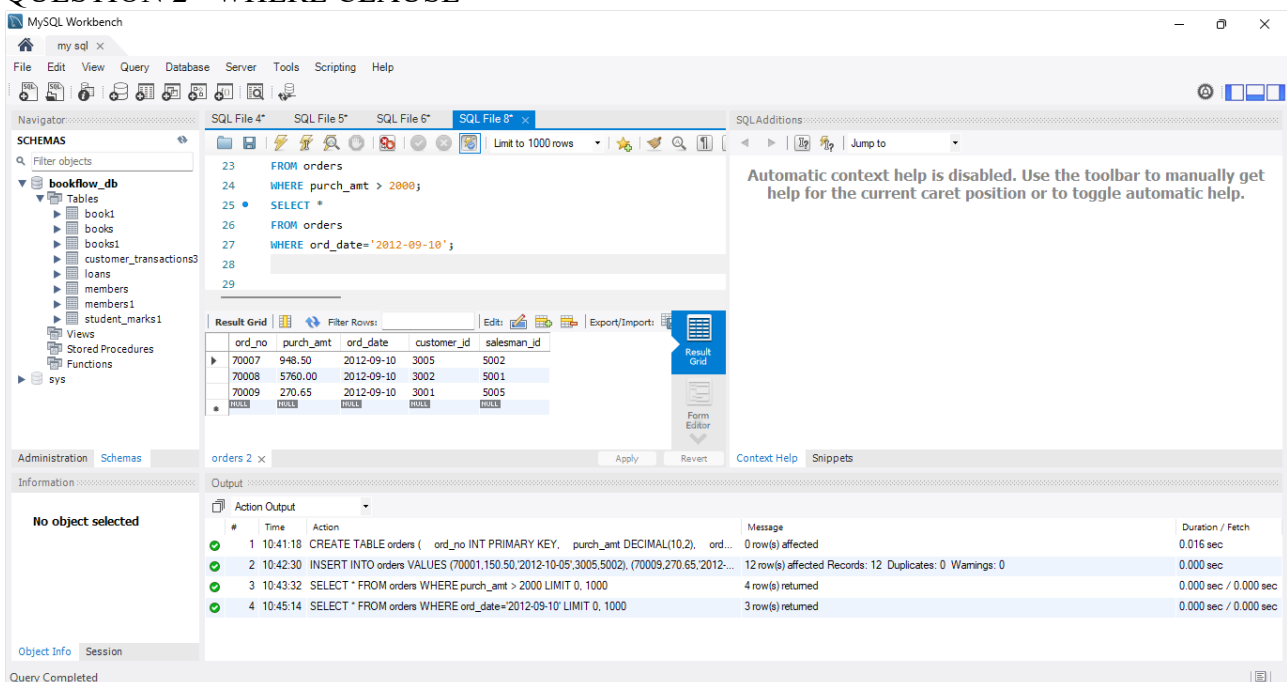
The 'Result Grid' shows the following data:

ord_no	purch_amt	ord_date	customer_id	salesman_id
70003	2480.40	2012-10-10	3009	5003
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001

The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	10:41:18	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ord...	0 row(s) affected	0.016 sec
2	10:42:30	INSERT INTO orders VALUES (70001,150.50,'2012-10-05',3005,5002), (70009,270.65,'2012...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0	0.000 sec
3	10:43:32	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec

QUESTION 2 - WHERE CLAUSE



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'bookflow_db' schema. The main editor window shows a SQL query in 'SQL File 8*':

```
23 FROM orders
24 WHERE purch_amt > 2000;
25 • SELECT *
26 FROM orders
27 WHERE ord_date='2012-09-10';
28
29
```

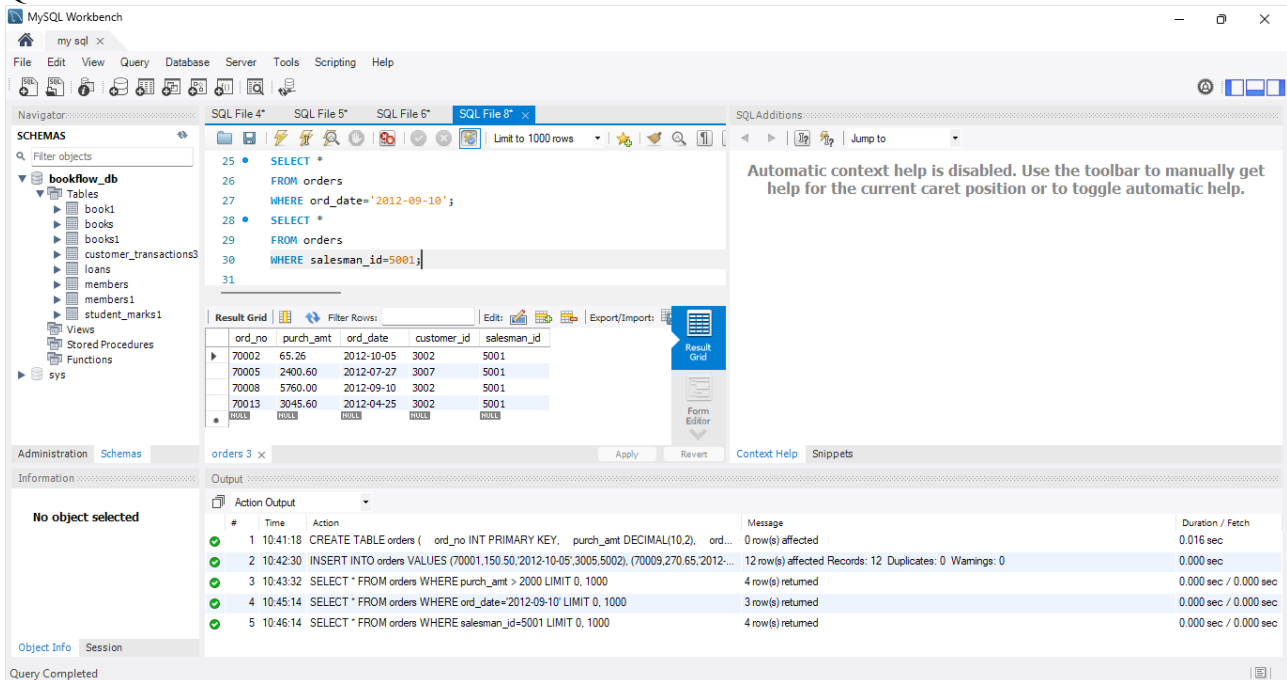
The 'Result Grid' shows the following data:

ord_no	purch_amt	ord_date	customer_id	salesman_id
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005

The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	10:41:18	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ord...	0 row(s) affected	0.016 sec
2	10:42:30	INSERT INTO orders VALUES (70001,150.50,'2012-10-05',3005,5002), (70009,270.65,'2012...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0	0.000 sec
3	10:43:32	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
4	10:45:14	SELECT * FROM orders WHERE ord_date='2012-09-10' LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec

QUESTION 3 - WHERE CLAUSE



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'bookflow_db' schema with tables like 'book1', 'books', 'books1', 'customer_transactions3', 'loans', 'members', 'members1', 'student_marks1', 'Views', 'Stored Procedures', and 'Functions'. The main editor window shows a SQL query in 'SQL File 8':

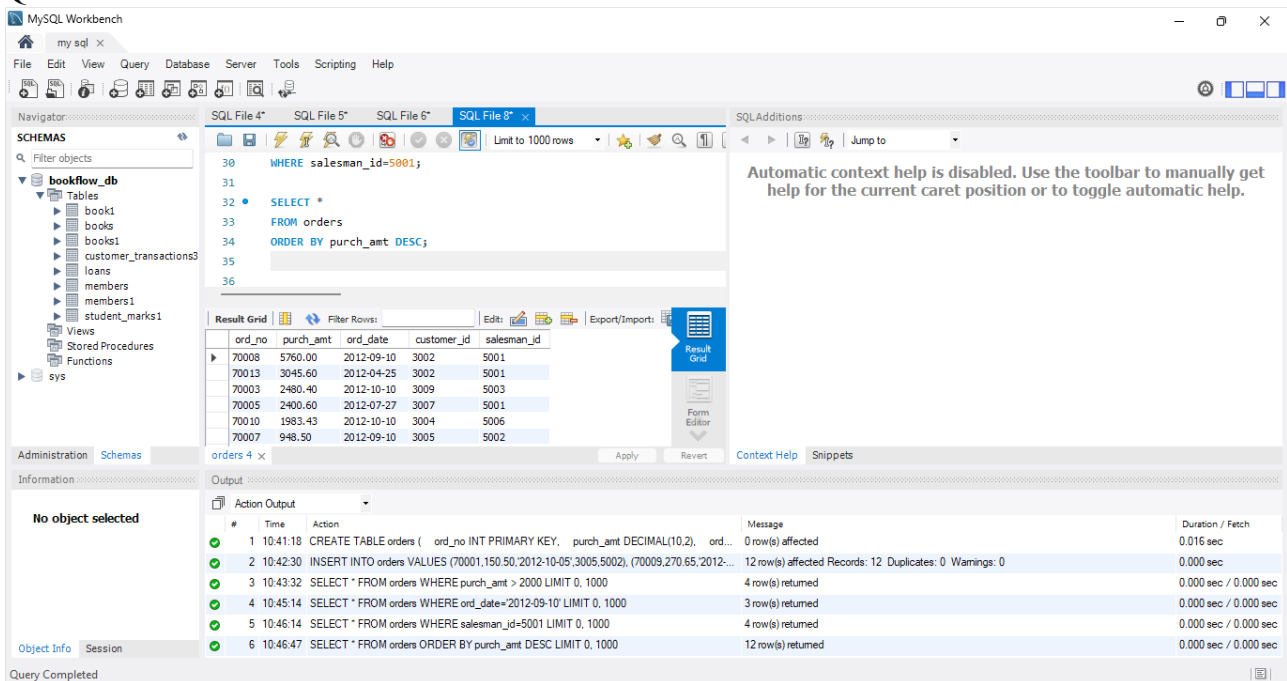
```
25 SELECT *
26 FROM orders
27 WHERE ord_date='2012-09-10';
28 SELECT *
29 FROM orders
30 WHERE salesman_id=5001;
```

The 'Result Grid' shows the results of the second query, displaying columns: ord_no, purch_amt, ord_date, customer_id, and salesman_id. The results are:

ord_no	purch_amt	ord_date	customer_id	salesman_id
70002	65.26	2012-10-05	3002	5001
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001

The 'Output' pane shows the execution log with 5 messages, including the execution of the two SELECT queries.

QUESTION 4 - ORDER BY



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'bookflow_db' schema. The main editor window shows a SQL query in 'SQL File 8':

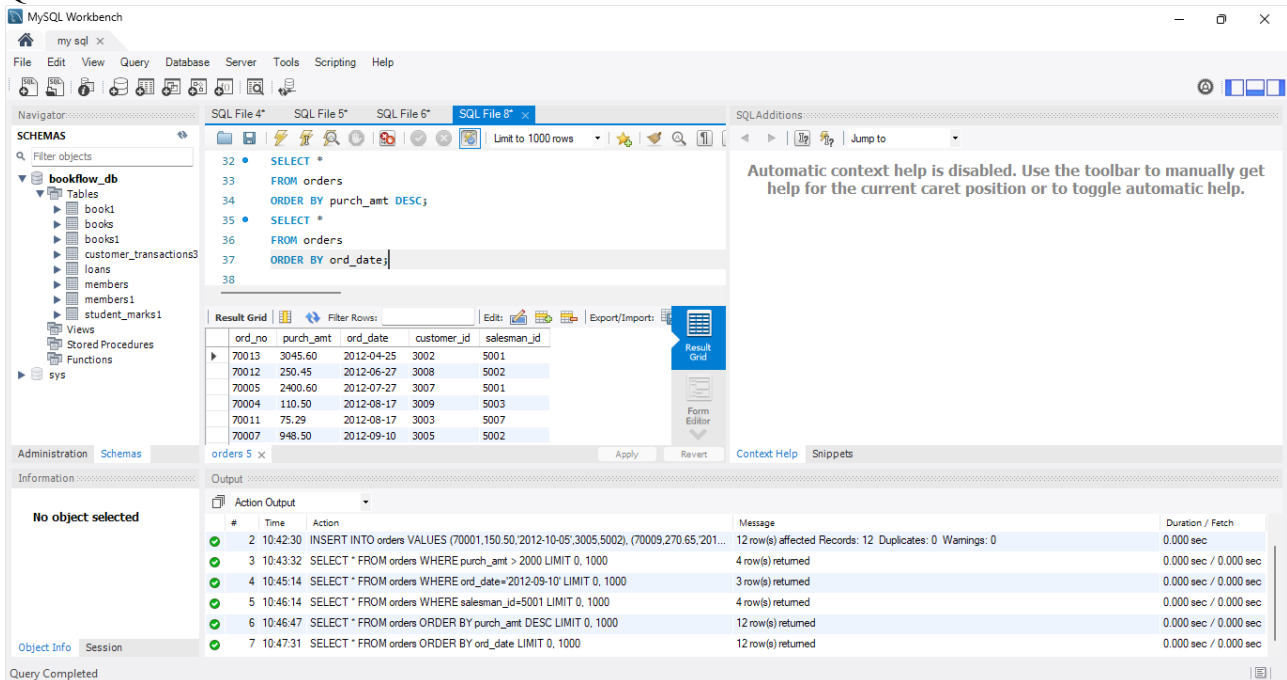
```
30 WHERE salesman_id=5001;
31
32 SELECT *
33 FROM orders
34 ORDER BY purch_amt DESC;
```

The 'Result Grid' shows the results of the second query, displaying columns: ord_no, purch_amt, ord_date, customer_id, and salesman_id. The results are sorted by purch_amt in descending order:

ord_no	purch_amt	ord_date	customer_id	salesman_id
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001
70003	2480.40	2012-10-10	3009	5003
70005	2400.60	2012-07-27	3007	5001
70010	1983.43	2012-10-10	3004	5006
70007	948.50	2012-09-10	3005	5002

The 'Output' pane shows the execution log with 6 messages, including the execution of the ORDER BY query.

QUESTION 5 - ORDER BY



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'bookflow_db' database with various tables and views. The main editor window contains the following SQL query:

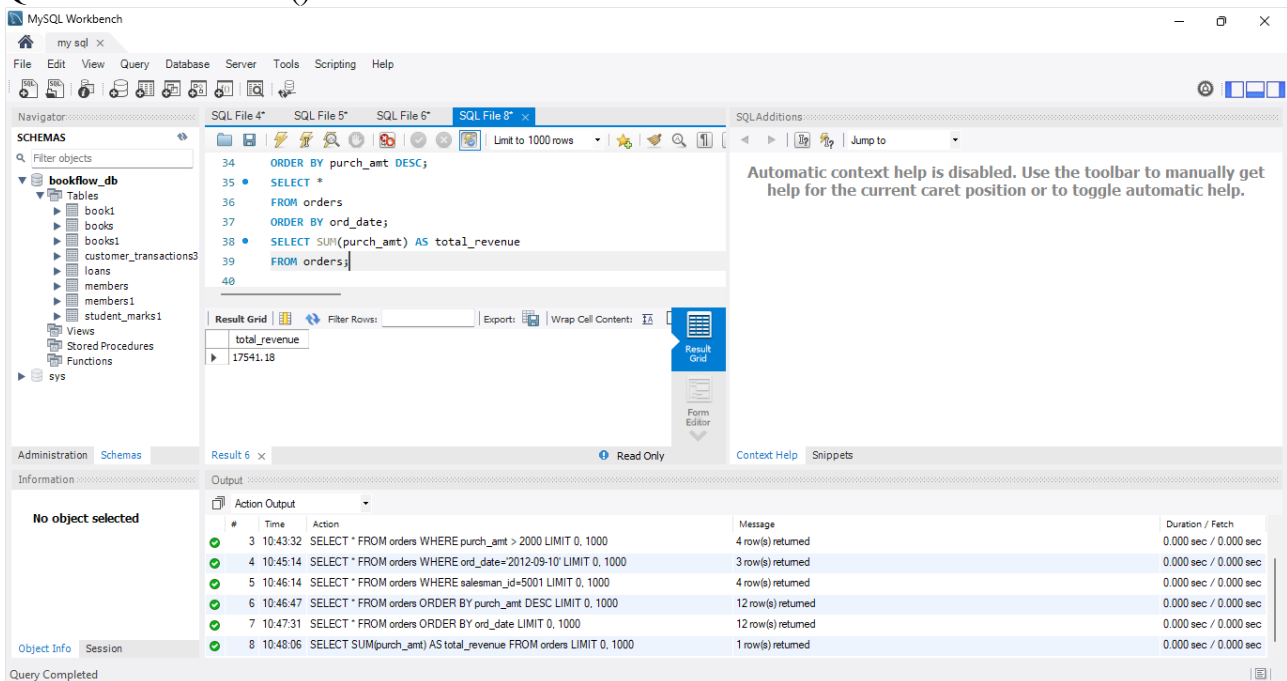
```
32 SELECT *
33 FROM orders
34 ORDER BY purch_amt DESC;
35 SELECT *
36 FROM orders
37 ORDER BY ord_date;
```

The 'Result Grid' shows the results of the second query, ordered by 'ord_date' in descending order:

ord_no	purch_amt	ord_date	customer_id	salesman_id
70013	3045.60	2012-04-25	3002	5001
70012	250.45	2012-06-27	3008	5002
70005	2400.60	2012-07-27	3007	5001
70004	110.50	2012-08-17	3009	5003
70011	75.29	2012-08-17	3003	5007
70007	948.50	2012-09-10	3005	5002

The 'Output' pane shows the execution log with 7 messages, including the final query execution.

QUESTION 6 – SUM()



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'bookflow_db' database. The main editor window contains the following SQL query:

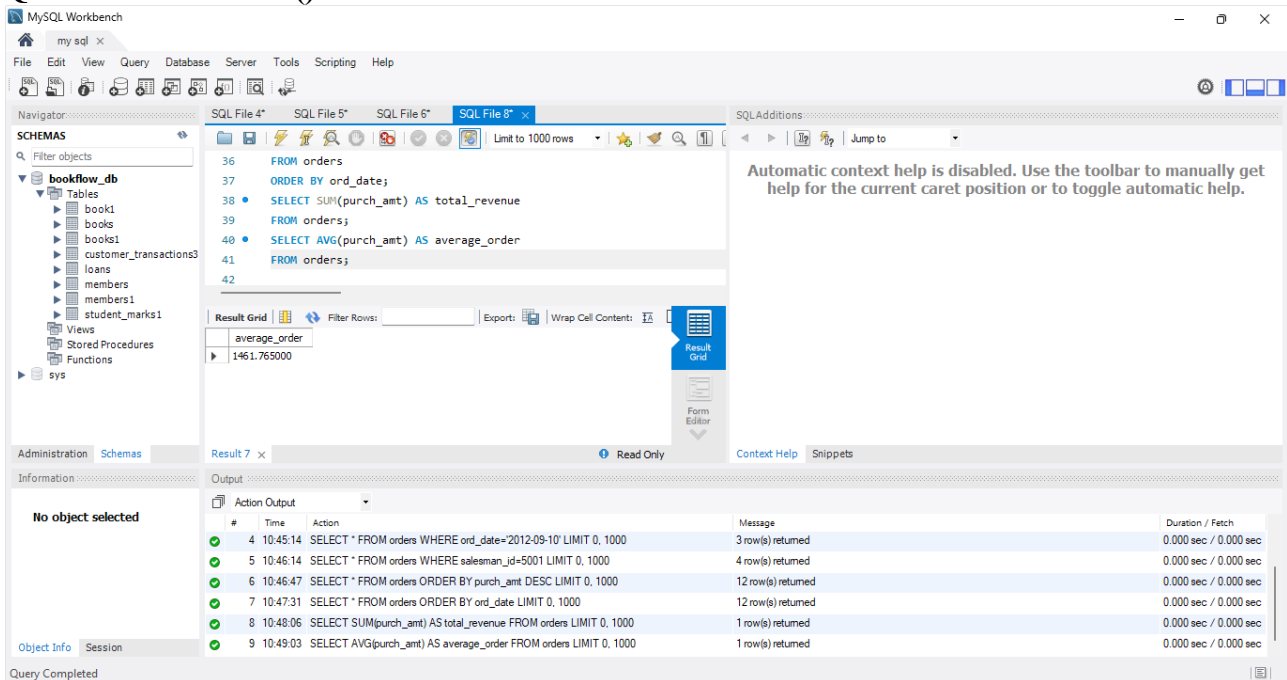
```
34 ORDER BY purch_amt DESC;
35 SELECT *
36 FROM orders
37 ORDER BY ord_date;
38 SELECT SUM(purch_amt) AS total_revenue
39 FROM orders;
```

The 'Result Grid' shows the results of the third query, which is a single row representing the total revenue:

total_revenue
17541.18

The 'Output' pane shows the execution log with 8 messages, including the final query execution.

QUESTION 7 – AVG()



The screenshot shows MySQL Workbench with a query editor containing SQL code for Question 7. The query calculates the average order amount from the 'orders' table. The result grid shows a single row with the value 1461.765000. The output pane shows the execution log with 9 messages.

```
36 FROM orders;
37 ORDER BY ord_date;
38 • SELECT SUM(purch_amt) AS total_revenue
39 FROM orders;
40 • SELECT AVG(purch_amt) AS average_order
41 FROM orders;
42
```

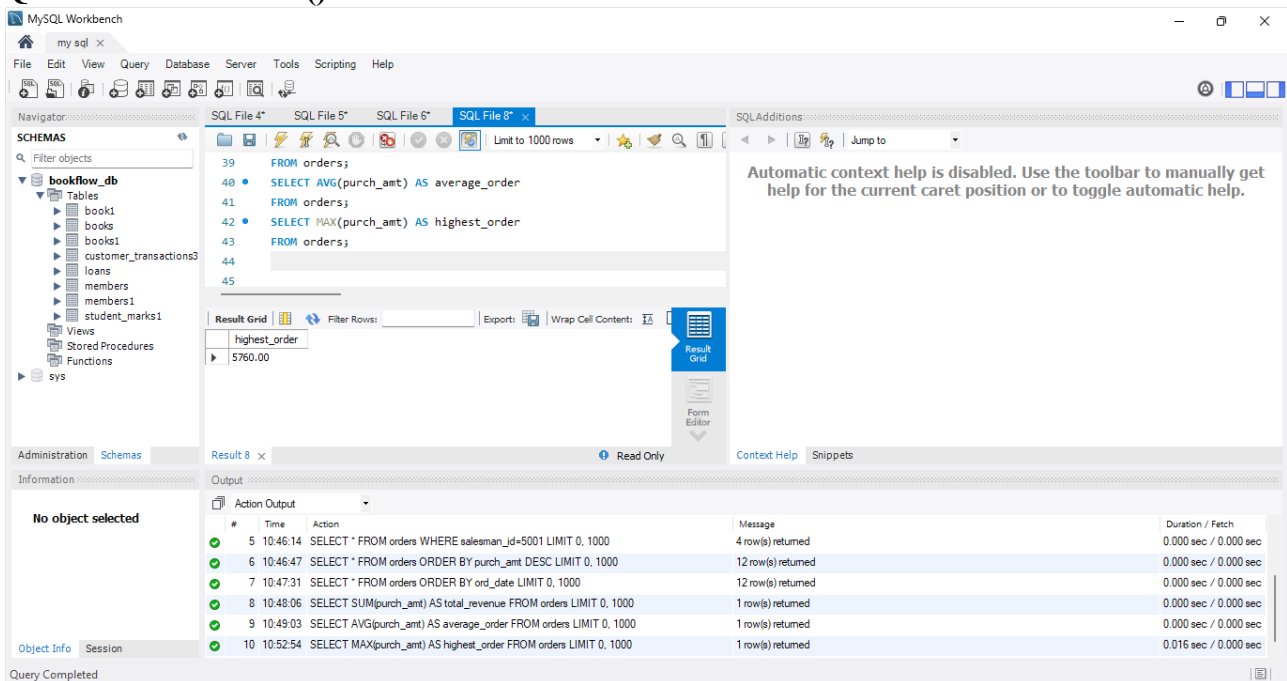
Result Grid

average_order
1461.765000

Output

#	Time	Action	Message	Duration / Fetch
4	10:45:14	SELECT * FROM orders WHERE ord_date='2012-09-10' LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
5	10:46:14	SELECT * FROM orders WHERE salesman_id=5001 LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
6	10:46:47	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned	0.000 sec / 0.000 sec
7	10:47:31	SELECT * FROM orders ORDER BY ord_date LIMIT 0, 1000	12 row(s) returned	0.000 sec / 0.000 sec
8	10:48:06	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
9	10:49:03	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

QUESTION 8 – MAX()



The screenshot shows MySQL Workbench with a query editor containing SQL code for Question 8. The query calculates the maximum order amount from the 'orders' table. The result grid shows a single row with the value 5760.00. The output pane shows the execution log with 10 messages.

```
39 FROM orders;
40 • SELECT AVG(purch_amt) AS average_order
41 FROM orders;
42 • SELECT MAX(purch_amt) AS highest_order
43 FROM orders;
44
45
```

Result Grid

highest_order
5760.00

Output

#	Time	Action	Message	Duration / Fetch
5	10:46:14	SELECT * FROM orders WHERE salesman_id=5001 LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
6	10:46:47	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned	0.000 sec / 0.000 sec
7	10:47:31	SELECT * FROM orders ORDER BY ord_date LIMIT 0, 1000	12 row(s) returned	0.000 sec / 0.000 sec
8	10:48:06	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
9	10:49:03	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
10	10:52:54	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec

QUESTION 9 – MIN()

MySQL Workbench interface showing a query execution for Question 9. The query is as follows:

```

41 FROM orders;
42 • SELECT MAX(purch_amt) AS highest_order
43 FROM orders;
44 • SELECT MIN(purch_amt) AS lowest_order
45 FROM orders;
46
47

```

The result grid shows the output of the query:

lowest_order
65.26

The output pane shows the following messages:

#	Time	Action	Message	Duration / Fetch
6	10:46:47	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned	0.000 sec / 0.000 sec
7	10:47:31	SELECT * FROM orders ORDER BY ord_date LIMIT 0, 1000	12 row(s) returned	0.000 sec / 0.000 sec
8	10:48:06	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
9	10:49:03	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
10	10:52:54	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
11	10:53:46	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

QUESTION 10 – COUNT()

MySQL Workbench interface showing a query execution for Question 10. The query is as follows:

```

43 FROM orders;
44 • SELECT MIN(purch_amt) AS lowest_order
45 FROM orders;
46 • SELECT COUNT(*) AS total_orders
47 FROM orders;
48
49

```

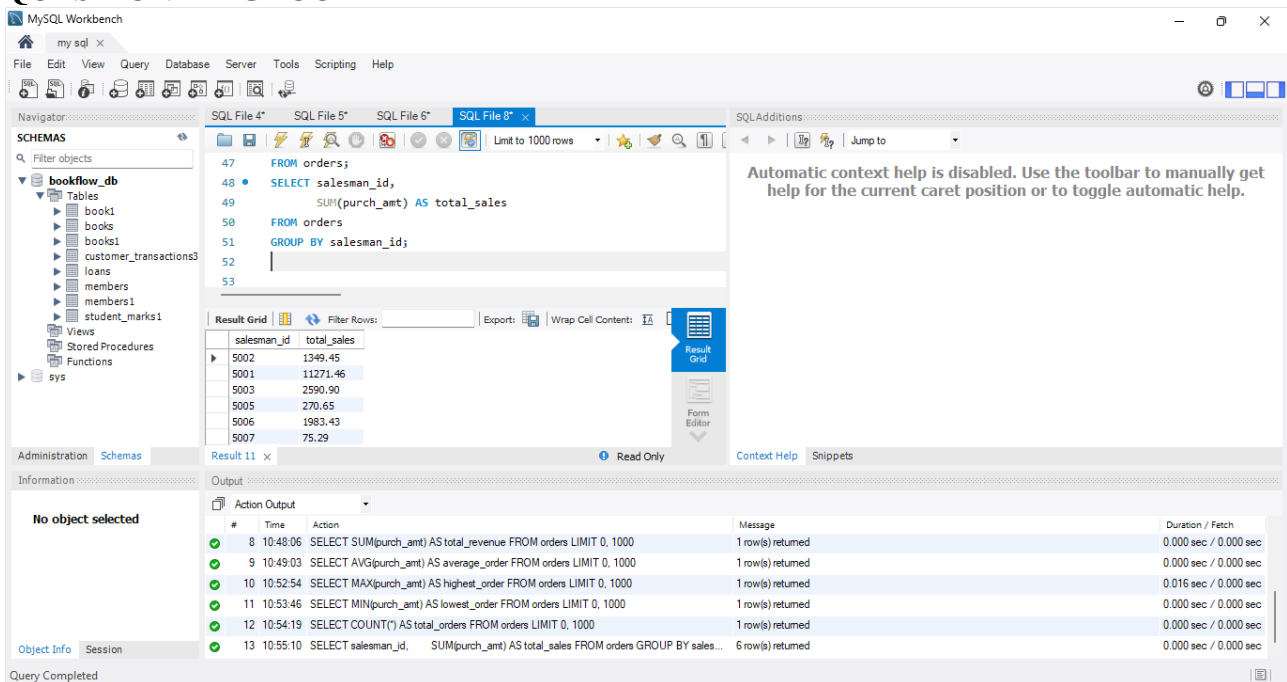
The result grid shows the output of the query:

total_orders
12

The output pane shows the following messages:

#	Time	Action	Message	Duration / Fetch
7	10:47:31	SELECT * FROM orders ORDER BY ord_date LIMIT 0, 1000	12 row(s) returned	0.000 sec / 0.000 sec
8	10:48:06	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
9	10:49:03	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
10	10:52:54	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
11	10:53:46	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
12	10:54:19	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

QUESTION 11 - GROUP BY



MySQL Workbench interface showing a query in SQL File 8*. The query is:

```
47 FROM orders;  
48 • SELECT salesman_id,  
49 SUM(purch_amt) AS total_sales  
50 FROM orders  
51 GROUP BY salesman_id;  
52  
53
```

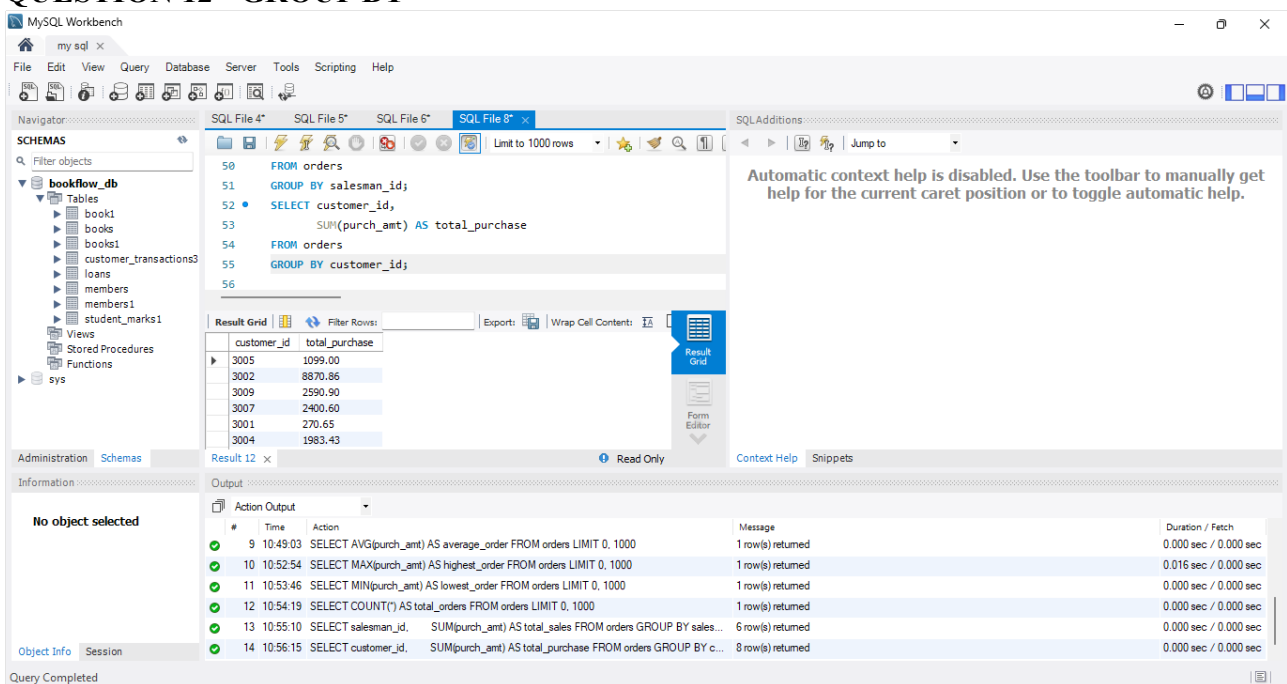
The Result Grid shows the following data:

salesman_id	total_sales
5002	1349.45
5001	11271.46
5003	2590.90
5005	270.65
5006	1983.43
5007	75.29

The Output pane shows the following messages:

#	Time	Action	Message	Duration / Fetch
8	10:48:06	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
9	10:49:03	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
10	10:52:54	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
11	10:53:46	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
12	10:54:19	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
13	10:55:10	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY sales...	6 row(s) returned	0.000 sec / 0.000 sec

QUESTION 12 - GROUP BY



MySQL Workbench interface showing a query in SQL File 8*. The query is:

```
50 FROM orders  
51 GROUP BY salesman_id;  
52 • SELECT customer_id,  
53 SUM(purch_amt) AS total_purchase  
54 FROM orders  
55 GROUP BY customer_id;  
56
```

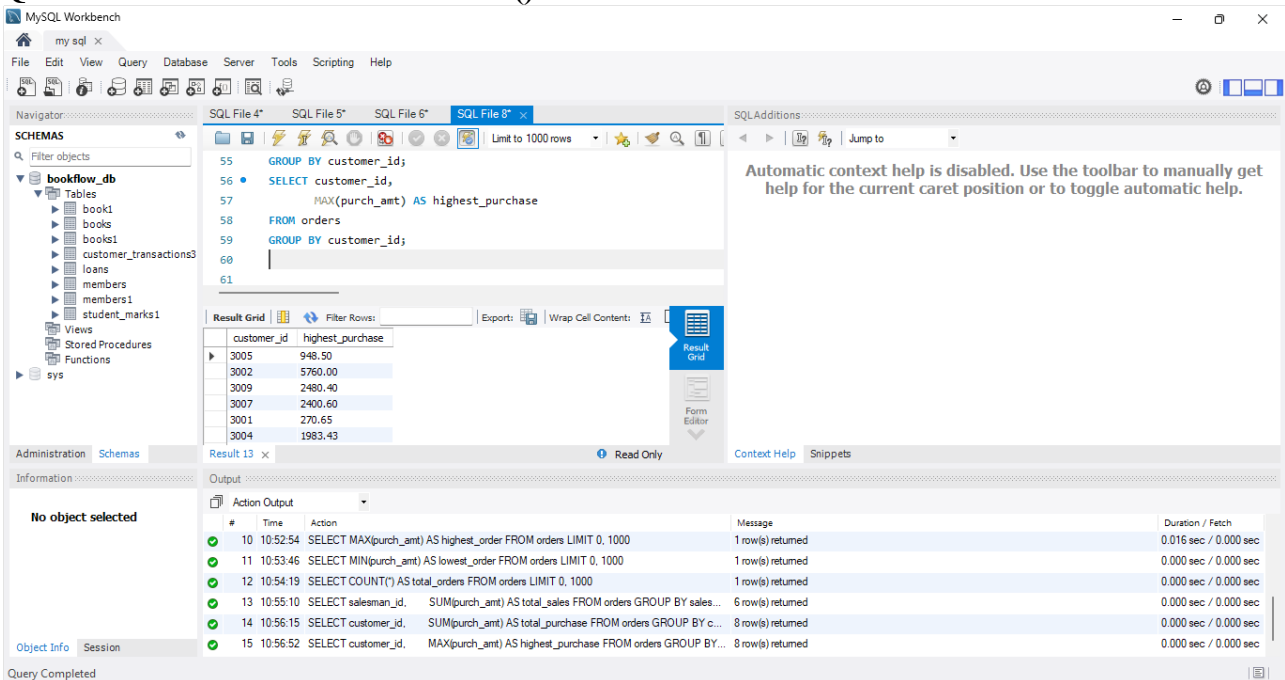
The Result Grid shows the following data:

customer_id	total_purchase
3005	1099.00
3002	8870.86
3009	2590.90
3007	2400.60
3001	270.65
3004	1983.43

The Output pane shows the following messages:

#	Time	Action	Message	Duration / Fetch
9	10:49:03	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
10	10:52:54	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
11	10:53:46	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
12	10:54:19	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
13	10:55:10	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY sales...	6 row(s) returned	0.000 sec / 0.000 sec
14	10:56:15	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY c...	8 row(s) returned	0.000 sec / 0.000 sec

QUESTION 13 - GROUP BY + MAX()



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'bookflow_db' schema with tables like 'book1', 'books', 'books1', 'customer_transactions3', 'loans', 'members', 'members1', 'student_marks1', 'Views', 'Stored Procedures', and 'Functions'. The main editor window contains the following SQL query:

```
55 GROUP BY customer_id;
56 SELECT customer_id,
57        MAX(purch_amt) AS highest_purchase
58 FROM orders
59 GROUP BY customer_id;
60
61
```

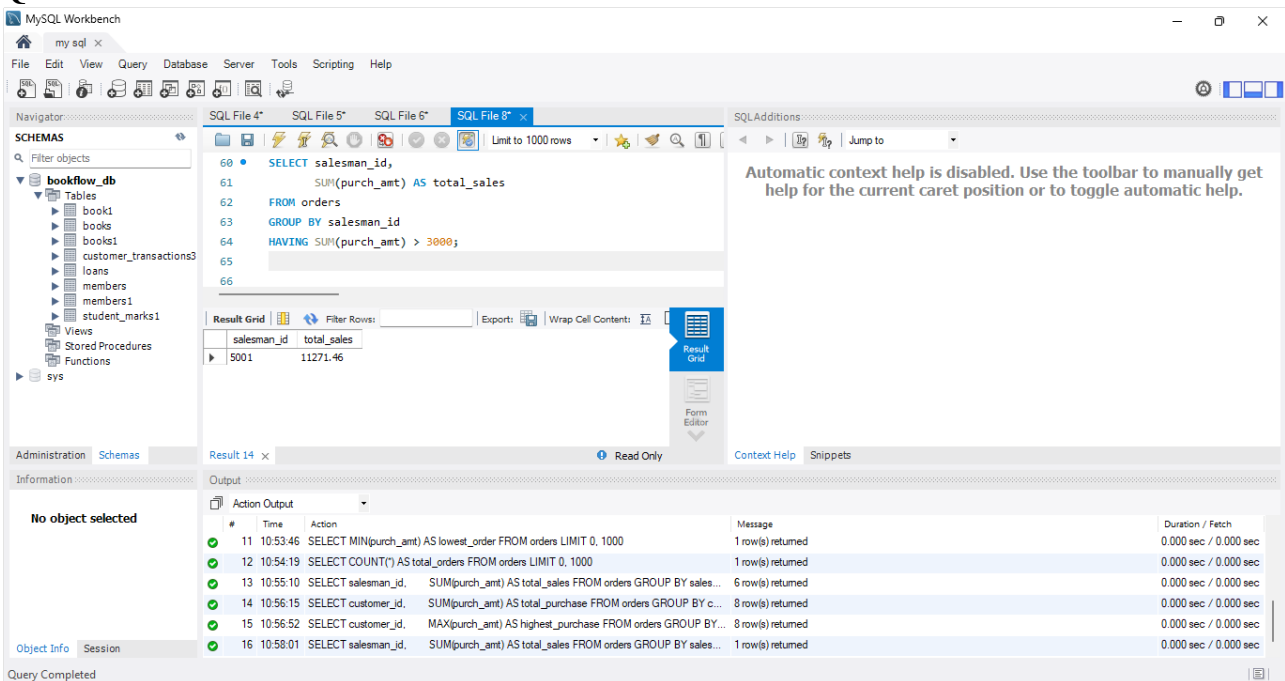
The 'Result Grid' shows the following data:

customer_id	highest_purchase
3005	948.50
3002	5760.00
3009	2480.40
3007	2400.60
3001	270.65
3004	1983.43

The 'Output' pane shows the execution log for Query 13:

#	Time	Action	Message	Duration / Fetch
10	10:52:54	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
11	10:53:46	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
12	10:54:19	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
13	10:55:10	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id	6 row(s) returned	0.000 sec / 0.000 sec
14	10:56:15	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id	8 row(s) returned	0.000 sec / 0.000 sec
15	10:56:52	SELECT customer_id, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY customer_id	8 row(s) returned	0.000 sec / 0.000 sec

QUESTION 14 - GROUP BY + HAVING



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'bookflow_db' schema. The main editor window contains the following SQL query:

```
60 SELECT salesman_id,
61        SUM(purch_amt) AS total_sales
62 FROM orders
63 GROUP BY salesman_id
64 HAVING SUM(purch_amt) > 3000;
65
66
```

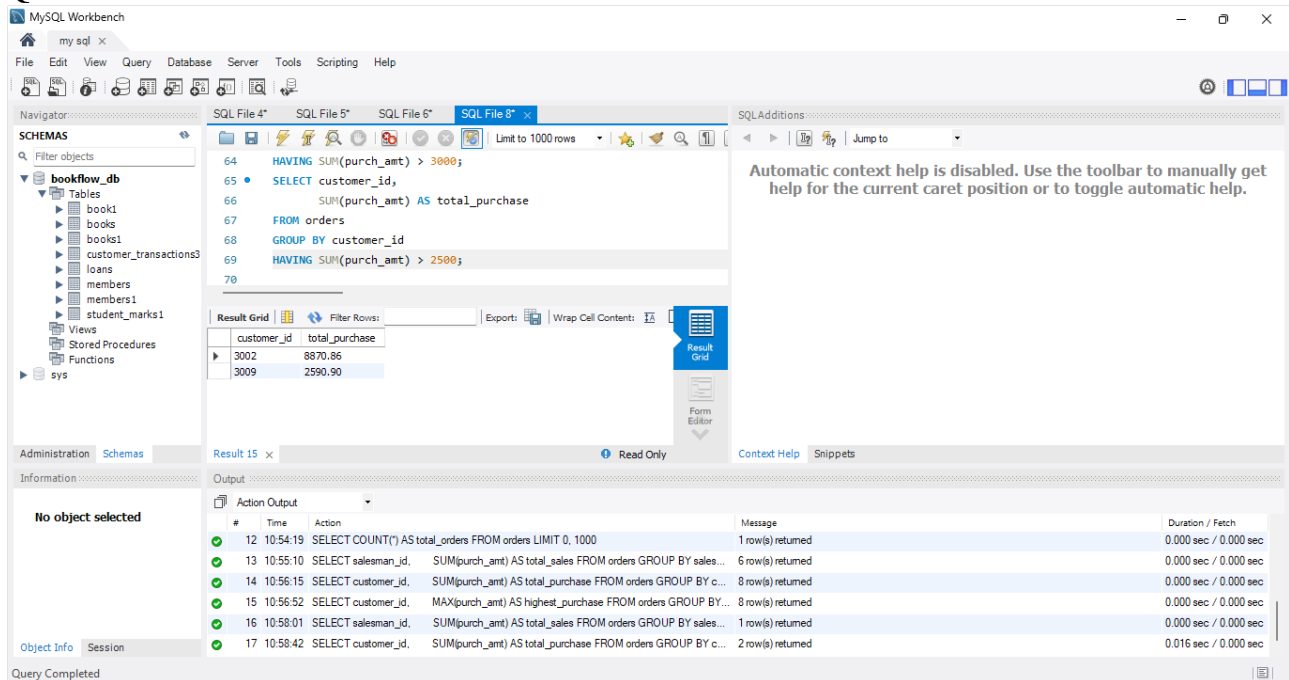
The 'Result Grid' shows the following data:

salesman_id	total_sales
5001	11271.46

The 'Output' pane shows the execution log for Query 14:

#	Time	Action	Message	Duration / Fetch
11	10:53:46	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
12	10:54:19	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
13	10:55:10	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id	6 row(s) returned	0.000 sec / 0.000 sec
14	10:56:15	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id	8 row(s) returned	0.000 sec / 0.000 sec
15	10:56:52	SELECT customer_id, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY customer_id	8 row(s) returned	0.000 sec / 0.000 sec
16	10:58:01	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id HAVING SUM(purch_amt) > 3000	1 row(s) returned	0.000 sec / 0.000 sec

QUESTION 15 - GROUP BY + HAVING



MySQL Workbench interface showing a SQL query in the SQL Editor. The query is:

```
64 HAVING SUM(purch_amt) > 3000;  
65 SELECT customer_id,  
66 SUM(purch_amt) AS total_purchase  
67 FROM orders  
68 GROUP BY customer_id  
69 HAVING SUM(purch_amt) > 2500;  
70
```

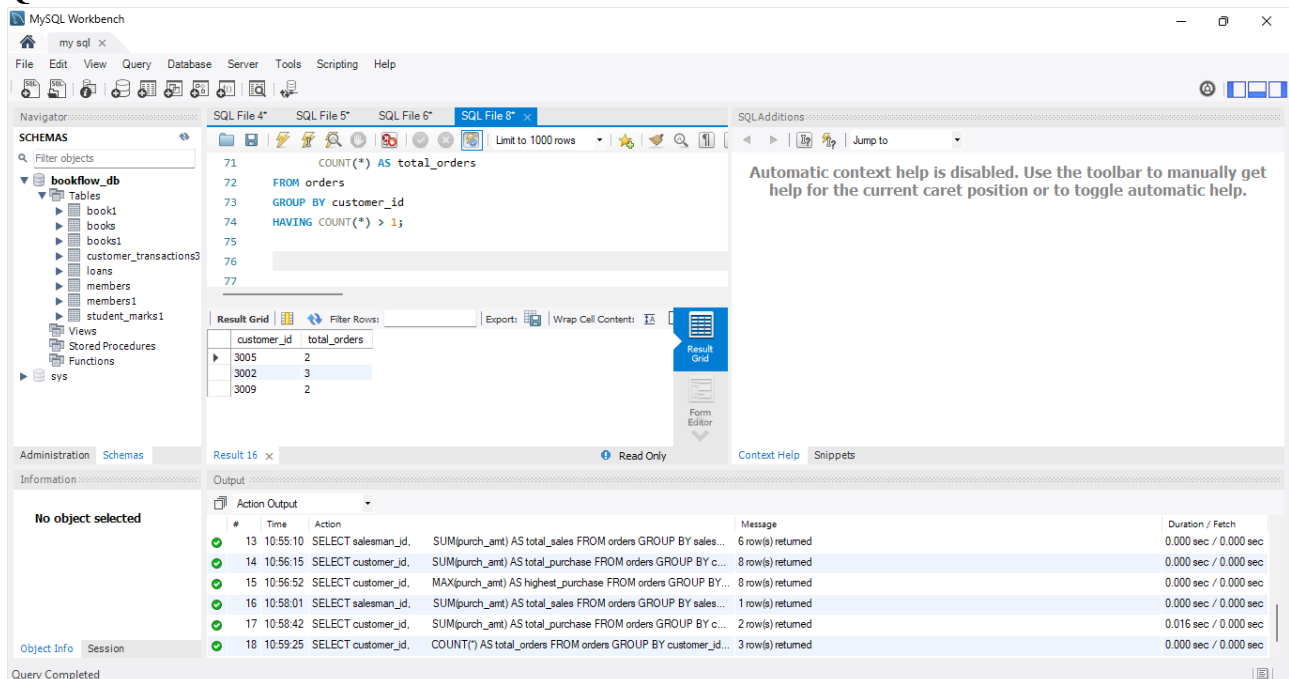
The Result Grid shows the following data:

customer_id	total_purchase
3002	8870.86
3009	2590.90

The Output pane shows the execution log with the following messages:

#	Time	Action	Message	Duration / Fetch
12	10:54:19	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
13	10:55:10	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id	6 row(s) returned	0.000 sec / 0.000 sec
14	10:56:15	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id	8 row(s) returned	0.000 sec / 0.000 sec
15	10:56:52	SELECT customer_id, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY customer_id	8 row(s) returned	0.000 sec / 0.000 sec
16	10:58:01	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id	1 row(s) returned	0.000 sec / 0.000 sec
17	10:58:42	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id	2 row(s) returned	0.016 sec / 0.000 sec

QUESTION 16 - GROUP BY + HAVING



MySQL Workbench interface showing a SQL query in the SQL Editor. The query is:

```
71 COUNT(*) AS total_orders  
72 FROM orders  
73 GROUP BY customer_id  
74 HAVING COUNT(*) > 1;  
75  
76  
77
```

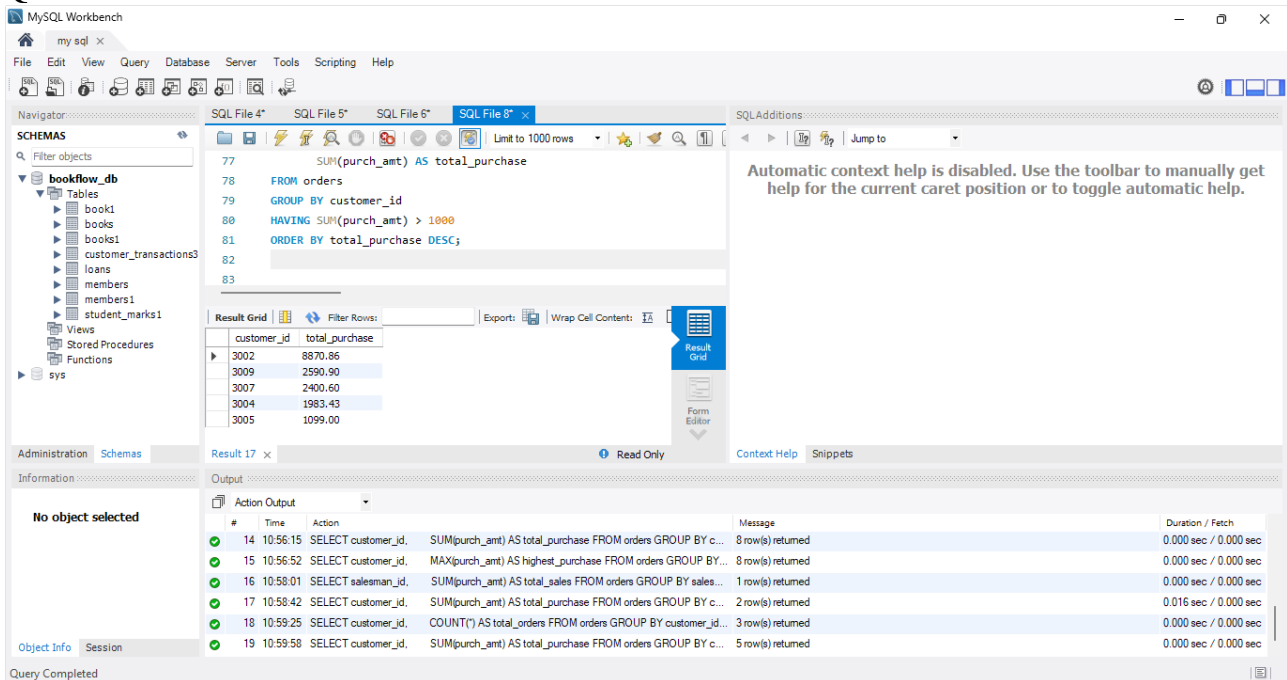
The Result Grid shows the following data:

customer_id	total_orders
3005	2
3002	3
3009	2

The Output pane shows the execution log with the following messages:

#	Time	Action	Message	Duration / Fetch
13	10:55:10	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id	6 row(s) returned	0.000 sec / 0.000 sec
14	10:56:15	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id	8 row(s) returned	0.000 sec / 0.000 sec
15	10:56:52	SELECT customer_id, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY customer_id	8 row(s) returned	0.000 sec / 0.000 sec
16	10:58:01	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id	1 row(s) returned	0.000 sec / 0.000 sec
17	10:58:42	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id	2 row(s) returned	0.016 sec / 0.000 sec
18	10:59:25	SELECT customer_id, COUNT(*) AS total_orders FROM orders GROUP BY customer_id	3 row(s) returned	0.000 sec / 0.000 sec

QUESTION 17 - GROUP BY + HAVING + ORDER BY



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
77 SUM(purch_amt) AS total_purchase
78 FROM orders
79 GROUP BY customer_id
80 HAVING SUM(purch_amt) > 1000
81 ORDER BY total_purchase DESC;
82
83
```

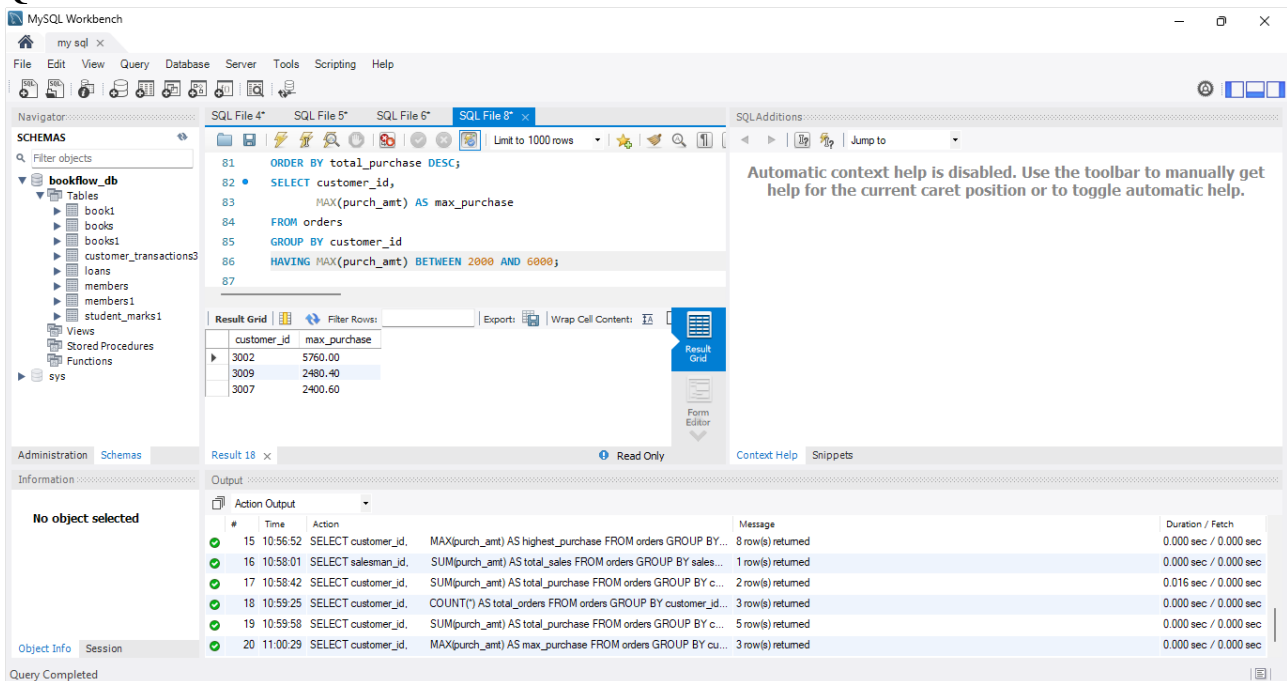
The Result Grid shows the following data:

customer_id	total_purchase
3002	8870.86
3009	2590.90
3007	2400.60
3004	1983.43
3005	1099.00

The Output pane shows the following log entries:

#	Time	Action	Message	Duration / Fetch
14	10:56:15	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id	8 row(s) returned	0.000 sec / 0.000 sec
15	10:56:52	SELECT customer_id, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY customer_id	8 row(s) returned	0.000 sec / 0.000 sec
16	10:58:01	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id	1 row(s) returned	0.000 sec / 0.000 sec
17	10:58:42	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id	2 row(s) returned	0.016 sec / 0.000 sec
18	10:59:25	SELECT customer_id, COUNT(*) AS total_orders FROM orders GROUP BY customer_id	3 row(s) returned	0.000 sec / 0.000 sec
19	10:59:58	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id	5 row(s) returned	0.000 sec / 0.000 sec

QUESTION 18 - BETWEEN + HAVING



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
81 ORDER BY total_purchase DESC;
82 SELECT customer_id,
83 MAX(purch_amt) AS max_purchase
84 FROM orders
85 GROUP BY customer_id
86 HAVING MAX(purch_amt) BETWEEN 2000 AND 6000;
87
```

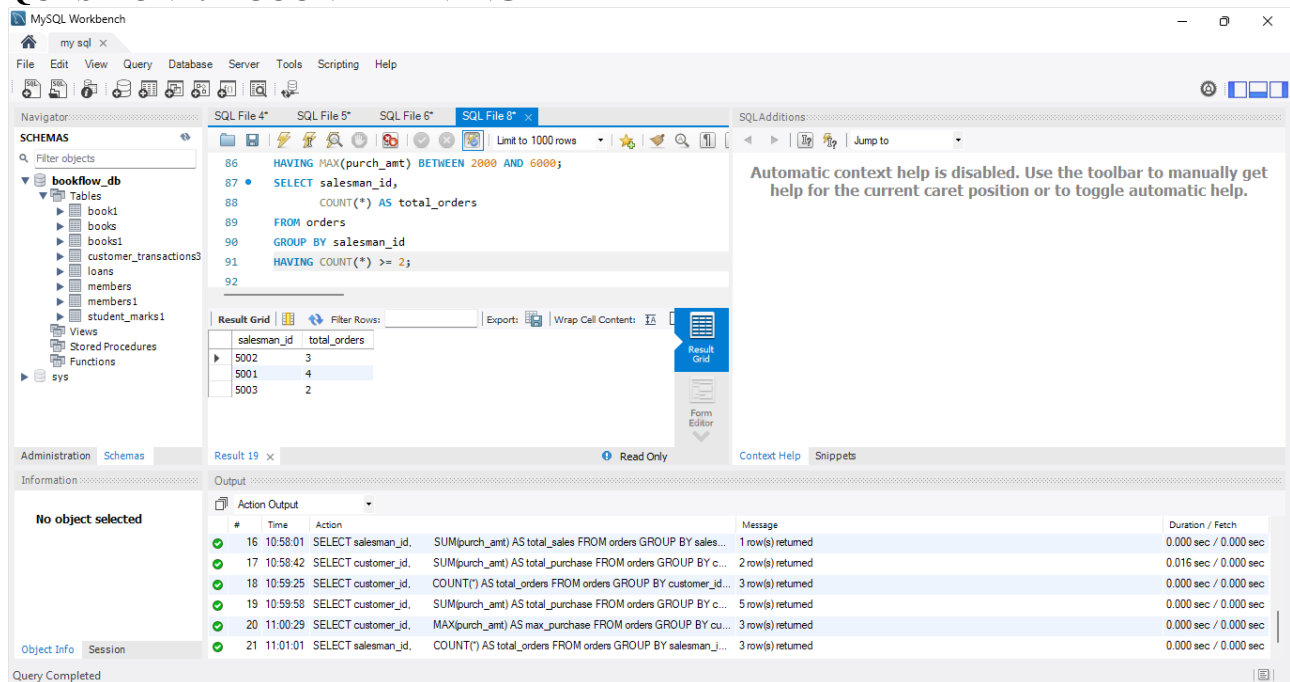
The Result Grid shows the following data:

customer_id	max_purchase
3002	5760.00
3009	2480.40
3007	2400.60

The Output pane shows the following log entries:

#	Time	Action	Message	Duration / Fetch
15	10:56:52	SELECT customer_id, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY customer_id	8 row(s) returned	0.000 sec / 0.000 sec
16	10:58:01	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id	1 row(s) returned	0.000 sec / 0.000 sec
17	10:58:42	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id	2 row(s) returned	0.016 sec / 0.000 sec
18	10:59:25	SELECT customer_id, COUNT(*) AS total_orders FROM orders GROUP BY customer_id	3 row(s) returned	0.000 sec / 0.000 sec
19	10:59:58	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id	5 row(s) returned	0.000 sec / 0.000 sec
20	11:00:29	SELECT customer_id, MAX(purch_amt) AS max_purchase FROM orders GROUP BY customer_id	3 row(s) returned	0.000 sec / 0.000 sec

QUESTION 19 - COUNT + HAVING



The screenshot shows the MySQL Workbench interface with a query editor containing the following SQL code:

```
86 HAVING MAX(purch_amt) BETWEEN 2000 AND 6000;  
87 SELECT salesman_id,  
88 COUNT(*) AS total_orders  
89 FROM orders  
90 GROUP BY salesman_id  
91 HAVING COUNT(*) >= 2;  
92
```

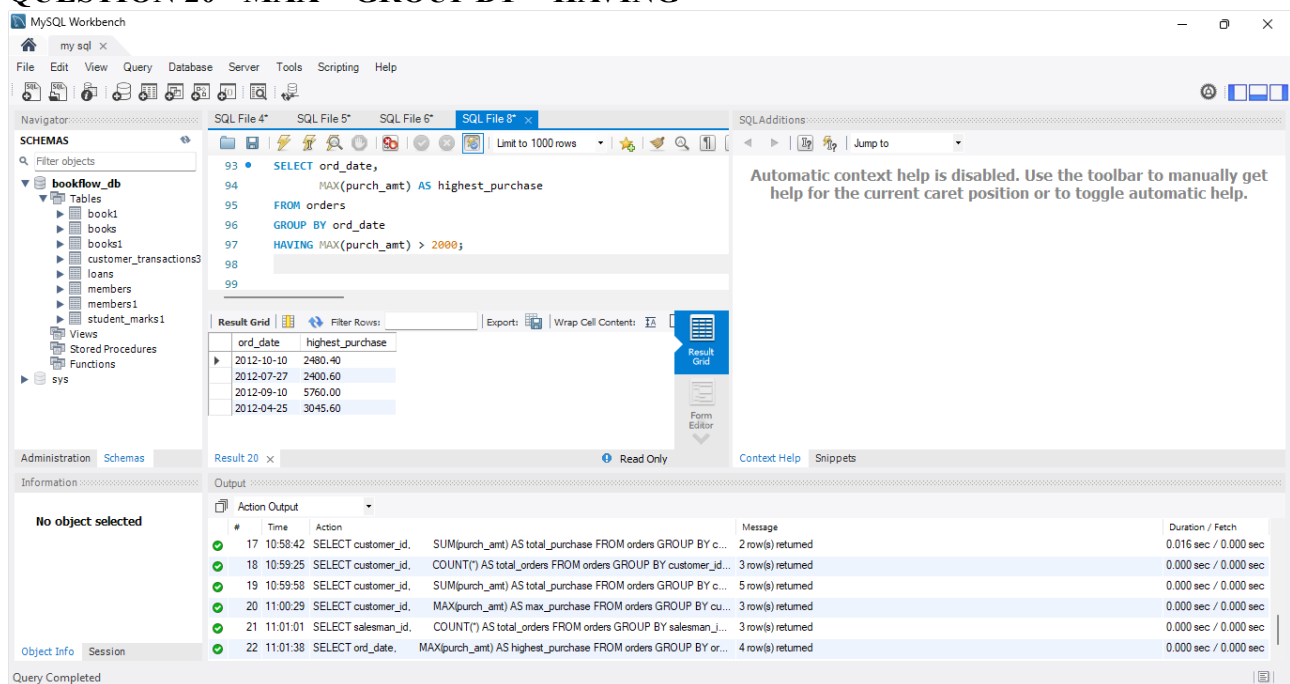
The result grid displays the following data:

salesman_id	total_orders
5002	3
5001	4
5003	2

The output pane shows the execution log with the following messages:

#	Time	Action	Message	Duration / Fetch
16	10:58:01	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY sales...	1 row(s) returned	0.000 sec / 0.000 sec
17	10:58:42	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY c...	2 row(s) returned	0.016 sec / 0.000 sec
18	10:59:25	SELECT customer_id, COUNT(*) AS total_orders FROM orders GROUP BY customer_id...	3 row(s) returned	0.000 sec / 0.000 sec
19	10:59:58	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY c...	5 row(s) returned	0.000 sec / 0.000 sec
20	11:00:29	SELECT customer_id, MAX(purch_amt) AS max_purchase FROM orders GROUP BY cu...	3 row(s) returned	0.000 sec / 0.000 sec
21	11:01:01	SELECT salesman_id, COUNT(*) AS total_orders FROM orders GROUP BY salesman_j...	3 row(s) returned	0.000 sec / 0.000 sec

QUESTION 20 - MAX + GROUP BY + HAVING



The screenshot shows the MySQL Workbench interface with a query editor containing the following SQL code:

```
93 SELECT ord_date,  
94 MAX(purch_amt) AS highest_purchase  
95 FROM orders  
96 GROUP BY ord_date  
97 HAVING MAX(purch_amt) > 2000;  
98  
99
```

The result grid displays the following data:

ord_date	highest_purchase
2012-10-10	2480.40
2012-07-27	2400.60
2012-09-10	5760.00
2012-04-25	3045.60

The output pane shows the execution log with the following messages:

#	Time	Action	Message	Duration / Fetch
17	10:58:42	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY c...	2 row(s) returned	0.016 sec / 0.000 sec
18	10:59:25	SELECT customer_id, COUNT(*) AS total_orders FROM orders GROUP BY customer_id...	3 row(s) returned	0.000 sec / 0.000 sec
19	10:59:58	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY c...	5 row(s) returned	0.000 sec / 0.000 sec
20	11:00:29	SELECT customer_id, MAX(purch_amt) AS max_purchase FROM orders GROUP BY cu...	3 row(s) returned	0.000 sec / 0.000 sec
21	11:01:01	SELECT salesman_id, COUNT(*) AS total_orders FROM orders GROUP BY salesman_j...	3 row(s) returned	0.000 sec / 0.000 sec
22	11:01:38	SELECT ord_date, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY or...	4 row(s) returned	0.000 sec / 0.000 sec

FINAL OUTPUTS:

MySQL Workbench

my sql x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 4* SQL File 5* SQL File 6* SQL File 8* SQLAdditions

SCHEMAS

Filter objects

bookflow_db

- Tables
 - book1
 - books
 - books1
 - customer_transactions3
 - loans
 - members
 - members1
 - student_marks1
- Views
- Stored Procedures
- Functions
- sys

Administration Schemas

Information

No object selected

Object Info Session

Query Completed

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:41:18	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ord...	0 row(s) affected	0.016 sec
2	10:42:30	INSERT INTO orders VALUES (70001,150.50,'2012-10-05',3005,5002), (70009,270.65,'2012-...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0	0.000 sec
3	10:43:32	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
4	10:45:14	SELECT * FROM orders WHERE ord_date='2012-09-10' LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
5	10:46:14	SELECT * FROM orders WHERE salesman_id=5001 LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
6	10:46:47	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned	0.000 sec / 0.000 sec
7	10:47:31	SELECT * FROM orders ORDER BY ord_date LIMIT 0, 1000	12 row(s) returned	0.000 sec / 0.000 sec
8	10:48:06	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
9	10:49:03	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
10	10:52:54	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
11	10:53:46	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
12	10:54:19	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
13	10:55:10	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesma...	6 row(s) returned	0.000 sec / 0.000 sec
14	10:56:15	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY cust...	8 row(s) returned	0.000 sec / 0.000 sec
15	10:56:52	SELECT customer_id, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY c...	8 row(s) returned	0.000 sec / 0.000 sec
16	10:58:01	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesma...	1 row(s) returned	0.000 sec / 0.000 sec
17	10:58:42	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY cust...	2 row(s) returned	0.016 sec / 0.000 sec
18	10:59:25	SELECT customer_id, COUNT(*) AS total_orders FROM orders GROUP BY customer_id ...	3 row(s) returned	0.000 sec / 0.000 sec
19	10:59:58	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY cust...	5 row(s) returned	0.000 sec / 0.000 sec
20	11:00:29	SELECT customer_id, MAX(purch_amt) AS max_purchase FROM orders GROUP BY cust...	3 row(s) returned	0.000 sec / 0.000 sec
21	11:01:01	SELECT salesman_id, COUNT(*) AS total_orders FROM orders GROUP BY salesman_id ...	3 row(s) returned	0.000 sec / 0.000 sec
22	11:01:38	SELECT ord_date, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY ord_...	4 row(s) returned	0.000 sec / 0.000 sec